Showing Invariance Compositionally for a Process Algebra for Network Protocols

Timothy Bourke[1,2], Robert J. van Glabbeek[3,4], and Peter Höfner[3,4]

[1] Inria Paris-Rocquencourt, France
[2] Ecole normale supérieure, Paris, France
[3] NICTA, Sydney, Australia
[4] Computer Science and Engineering, UNSW, Sydney, Australia

Abstract. This paper presents the mechanization of a process algebra for Mobile Ad hoc Networks and Wireless Mesh Networks, and the development of a compositional framework for proving invariant properties. Mechanizing the core process algebra in Isabelle/HOL is relatively standard, but its layered structure necessitates special treatment. The control states of reactive processes, such as nodes in a network, are modelled by terms of the process algebra. We propose a technique based on these terms to streamline proofs of inductive invariance. This is not sufficient, however, to state and prove invariants that relate states across multiple processes (entire networks). To this end, we propose a novel compositional technique for lifting global invariants stated at the level of individual nodes to networks of nodes.

1 Introduction and Related Work

The Algebra for Wireless Networks (AWN) is a process algebra developed for modelling and analysing protocols for Mobile Ad hoc Networks (MANETs) and Wireless Mesh Networks (WMNs) [6, §4]. This paper reports on both its mechanization in Isabelle/HOL [15] and the development of a compositional framework for showing invariant properties of models.[1] The techniques we describe are a response to problems encountered during the mechanization of a model and proof—presented elsewhere [4]—of an RFC-standard for routing protocols. Despite the existence of extensive research on related problems [18] and several mechanized frameworks for reactive systems [5,10,14], we are not aware of other solutions that allow the compositional statement and proof of properties relating the states of different nodes in a message-passing model—at least not within the strictures imposed by an Interactive Theorem Prover (ITP).

But is there really any need for yet another process algebra and associated framework? AWN provides a unique mix of communication primitives and a treatment of data structures that are essential for studying MANET and WMN protocols with dynamic topologies and sophisticated routing logic [6, §1]. It supports communication primitives for one-to-one (unicast), one-to-many (groupcast), and one-to-all (broadcast) message passing. AWN comprises distinct layers

[1] The Isabelle/HOL source files can be found in the Archive of Formal Proofs [3].

G. Klein and R. Gamboa (Eds.): ITP 2014, LNAI 8558, pp. 144–159, 2014.
© Springer International Publishing Switzerland 2014

for expressing the structure of nodes and networks. We exploit this structure, but we also expect the techniques proposed in Sections 3 and 4 to apply to similar layered modelling languages. Besides this, our work differs from other mechanizations for verifying reactive systems, like UNITY [10], TLA$^+$ [5], or I/O Automata [14] (from which we drew the most inspiration), in its explicit treatment of control states, in the form of process algebra terms, as distinct from data states. In this respect, our approach is close to that of Isabelle/Circus [7], but it differs in (1) the treatment of operators for composing nodes, which we model directly as functions on automata, (2) the treatment of recursive invocations, which we do not permit, and (3) our inclusion of a framework for compositional proofs. Other work in ITPs focuses on showing traditional properties of process algebras, like, for instance, the treatment of binders [1], that bisimulation equivalence is a congruence [9,11], or properties of fix-point induction [20], while we focus on what has been termed 'proof methodology' [8], and develop a compositional method for showing correctness properties of protocols specified in a process algebra. Alternatively, Paulson's inductive approach [16] can be applied to show properties of protocols specified with less generic infrastructure. But we think it to be better suited to systems specified in a 'declarative' style as opposed to the strongly operational models we consider.

Structure and contributions. Section 2 describes the mechanization of AWN. The basic definitions are routine but the layered structure of the language and the treatment of operators on networks as functions on automata are relatively novel and essential to understanding later sections. Section 3 describes our mechanization of the theory of inductive invariants, closely following [13]. We exploit the structure of AWN to generate verification conditions corresponding to those of pen-and-paper proofs [6, §7]. Section 4 presents a compositional technique for stating and proving invariants that relate states across multiple nodes. Basically, we substitute 'open' Structural Operational Semantics (SOS) rules over the global state for the standard rules over local states (Section 4.1), show the property over a single sequential process (Section 4.2), 'lift' it successively over layers that model message queueing and network communication (Section 4.3), and, ultimately, 'transfer' it to the original model (Section 4.4).

2 The Process Algebra AWN

AWN comprises five layers [6, §4]. We treat each layer as an automaton with states of a specific form and a given set of transition rules. We describe the layers from the bottom up over the following sections.

2.1 Sequential Processes

Sequential processes are used to encode protocol logic. Each is modelled by a *(recursive) specification* Γ of type $'\mathsf{p} \Rightarrow ('\mathsf{s}, '\mathsf{p}, '\mathsf{l})\ \mathsf{seqp}$, which maps process names of type $'\mathsf{p}$ to terms of type $('\mathsf{s}, '\mathsf{p}, '\mathsf{l})\ \mathsf{seqp}$, also parameterized by $'\mathsf{s}$, data states, and $'\mathsf{l}$, labels. States of sequential processes have the form (ξ, p) where ξ is a data state of type $'\mathsf{s}$ and p is a control term of type $('\mathsf{s}, '\mathsf{p}, '\mathsf{l})\ \mathsf{seqp}$.

$\{\mathsf{l}\}[\![\mathsf{u}]\!]\ \mathsf{p}$	$\mathsf{'l} \Rightarrow (\mathsf{'s} \Rightarrow \mathsf{'s}) \Rightarrow (\mathsf{'s}, \mathsf{'p}, \mathsf{'l})\ \mathsf{seqp} \Rightarrow (\mathsf{'s}, \mathsf{'p}, \mathsf{'l})\ \mathsf{seqp}$
$\{\mathsf{l}\}\langle \mathsf{g} \rangle\ \mathsf{p}$	$\mathsf{'l} \Rightarrow (\mathsf{'s} \Rightarrow \mathsf{'s\ set}) \Rightarrow (\mathsf{'s}, \mathsf{'p}, \mathsf{'l})\ \mathsf{seqp} \Rightarrow (\mathsf{'s}, \mathsf{'p}, \mathsf{'l})\ \mathsf{seqp}$
$\{\mathsf{l}\}\mathsf{unicast}(\mathsf{s}_{ip}, \mathsf{s}_{msg})\ .\ \mathsf{p} \triangleright \mathsf{q}$	$\mathsf{'l} \Rightarrow (\mathsf{'s} \Rightarrow \mathsf{ip}) \Rightarrow (\mathsf{'s} \Rightarrow \mathsf{msg}) \Rightarrow (\mathsf{'s}, \mathsf{'p}, \mathsf{'l})\ \mathsf{seqp} \Rightarrow (\mathsf{'s}, \mathsf{'p}, \mathsf{'l})\ \mathsf{seqp} \Rightarrow (\mathsf{'s}, \mathsf{'p}, \mathsf{'l})\ \mathsf{seqp}$
$\{\mathsf{l}\}\mathsf{broadcast}(\mathsf{s}_{msg})\ .\ \mathsf{p}$	$\mathsf{'l} \Rightarrow (\mathsf{'s} \Rightarrow \mathsf{msg}) \Rightarrow (\mathsf{'s}, \mathsf{'p}, \mathsf{'l})\ \mathsf{seqp} \Rightarrow (\mathsf{'s}, \mathsf{'p}, \mathsf{'l})\ \mathsf{seqp}$
$\{\mathsf{l}\}\mathsf{groupcast}(\mathsf{s}_{ips}, \mathsf{s}_{msg})\ .\ \mathsf{p}$	$\mathsf{'l} \Rightarrow (\mathsf{'s} \Rightarrow \mathsf{ip\ set}) \Rightarrow (\mathsf{'s} \Rightarrow \mathsf{msg}) \Rightarrow (\mathsf{'s}, \mathsf{'p}, \mathsf{'l})\ \mathsf{seqp} \Rightarrow (\mathsf{'s}, \mathsf{'p}, \mathsf{'l})\ \mathsf{seqp}$
$\{\mathsf{l}\}\mathsf{send}(\mathsf{s}_{msg})\ .\ \mathsf{p}$	$\mathsf{'l} \Rightarrow (\mathsf{'s} \Rightarrow \mathsf{msg}) \Rightarrow (\mathsf{'s}, \mathsf{'p}, \mathsf{'l})\ \mathsf{seqp} \Rightarrow (\mathsf{'s}, \mathsf{'p}, \mathsf{'l})\ \mathsf{seqp}$
$\{\mathsf{l}\}\mathsf{receive}(\mathsf{u}_{msg})\ .\ \mathsf{p}$	$\mathsf{'l} \Rightarrow (\mathsf{msg} \Rightarrow \mathsf{'s} \Rightarrow \mathsf{'s}) \Rightarrow (\mathsf{'s}, \mathsf{'p}, \mathsf{'l})\ \mathsf{seqp} \Rightarrow (\mathsf{'s}, \mathsf{'p}, \mathsf{'l})\ \mathsf{seqp}$
$\{\mathsf{l}\}\mathsf{deliver}(\mathsf{s}_{data})\ .\ \mathsf{p}$	$\mathsf{'l} \Rightarrow (\mathsf{'s} \Rightarrow \mathsf{data}) \Rightarrow (\mathsf{'s}, \mathsf{'p}, \mathsf{'l})\ \mathsf{seqp} \Rightarrow (\mathsf{'s}, \mathsf{'p}, \mathsf{'l})\ \mathsf{seqp}$
$\mathsf{p}_1 \oplus \mathsf{p}_2$	$(\mathsf{'s}, \mathsf{'p}, \mathsf{'l})\ \mathsf{seqp} \Rightarrow (\mathsf{'s}, \mathsf{'p}, \mathsf{'l})\ \mathsf{seqp} \Rightarrow (\mathsf{'s}, \mathsf{'p}, \mathsf{'l})\ \mathsf{seqp}$
$\mathsf{call}(\mathsf{pn})$	$\mathsf{'p} \Rightarrow (\mathsf{'s}, \mathsf{'p}, \mathsf{'l})\ \mathsf{seqp}$

(a) Term constructors for $(\mathsf{'s}, \mathsf{'p}, \mathsf{'l})\ \mathsf{seqp}$.

$$\frac{\xi' = \mathsf{u}\ \xi}{((\xi, \{\mathsf{l}\}[\![\mathsf{u}]\!]\ \mathsf{p}), \tau, (\xi', \mathsf{p})) \in \mathsf{seqp\text{-}sos}\ \Gamma} \qquad \frac{((\xi, \mathsf{p}), \mathsf{a}, (\xi', \mathsf{p}')) \in \mathsf{seqp\text{-}sos}\ \Gamma}{((\xi, \mathsf{p} \oplus \mathsf{q}), \mathsf{a}, (\xi', \mathsf{p}')) \in \mathsf{seqp\text{-}sos}\ \Gamma}$$

$$\frac{((\xi, \Gamma\ \mathsf{pn}), \mathsf{a}, (\xi', \mathsf{p}')) \in \mathsf{seqp\text{-}sos}\ \Gamma}{((\xi, \mathsf{call}(\mathsf{pn})), \mathsf{a}, (\xi', \mathsf{p}')) \in \mathsf{seqp\text{-}sos}\ \Gamma} \qquad \frac{((\xi, \mathsf{q}), \mathsf{a}, (\xi', \mathsf{q}')) \in \mathsf{seqp\text{-}sos}\ \Gamma}{((\xi, \mathsf{p} \oplus \mathsf{q}), \mathsf{a}, (\xi', \mathsf{q}')) \in \mathsf{seqp\text{-}sos}\ \Gamma}$$

$$((\xi, \{\mathsf{l}\}\mathsf{unicast}(\mathsf{s}_{ip}, \mathsf{s}_{msg})\ .\ \mathsf{p} \triangleright \mathsf{q}), \mathsf{unicast}\ (\mathsf{s}_{ip}\ \xi)\ (\mathsf{s}_{msg}\ \xi), (\xi, \mathsf{p})) \in \mathsf{seqp\text{-}sos}\ \Gamma$$

$$((\xi, \{\mathsf{l}\}\mathsf{unicast}(\mathsf{s}_{ip}, \mathsf{s}_{msg})\ .\ \mathsf{p} \triangleright \mathsf{q}), \neg\mathsf{unicast}\ (\mathsf{s}_{ip}\ \xi), (\xi, \mathsf{q})) \in \mathsf{seqp\text{-}sos}\ \Gamma$$

(b) SOS rules for sequential processes: examples from seqp-sos.

Fig. 1. Sequential processes: terms and semantics

Process terms are built from the constructors that are shown with their types[2] in Figure 1a. The inductive set seqp-sos, shown partially in Figure 1b, contains one or two SOS rules for each constructor. It is parameterized by a specification Γ and relates triples of source states, actions, and destination states.

The 'prefix' constructors are each labelled with an $\{\mathsf{l}\}$. Labels are used to strengthen invariants when a property is only true in or between certain states; they have no influence on control flow (unlike in [13]). The prefix constructors are *assignment*, $\{\mathsf{l}\}[\![\mathsf{u}]\!]\ \mathsf{p}$, which transforms the data state deterministically according to the function u and performs a τ action, as shown in Figure 1b; *guard/bind*, $\{\mathsf{l}\}\langle \mathsf{g} \rangle\ \mathsf{p}$, with which we encode both guards, $\langle \lambda\xi.\ \mathsf{if}\ \mathsf{g}\ \xi\ \mathsf{then}\ \{\xi\}\ \mathsf{else}\ \emptyset \rangle\ \mathsf{p}$, and variable bindings, as in $\langle \lambda\xi.\ \{\xi(\!|\mathsf{no} := \mathsf{n}|\!) \mid \mathsf{n} < 5\} \rangle\ \mathsf{p}$;[3] *network synchronizations*, receive/unicast/broadcast/groupcast, of which the rules for unicast are characteristic and shown in Figure 1b—the environment decides between a successful unicast i m and an unsuccessful ¬unicast i; and, *internal communications*, send/receive/deliver.

The other constructors are unlabelled and serve to 'glue' processes together: *choice*, $\mathsf{p}_1 \oplus \mathsf{p}_2$, takes the union of two transition sets; and, *call*, call(pn), affixes a term from the specification (Γ pn). The rules for both are shown in Figure 1b.

[2] Leading abstractions are omitted, for example, $\lambda\mathsf{l}\ \mathsf{fa}\ \mathsf{p}.\ \{\mathsf{l}\}[\![\mathsf{u}]\!]\ \mathsf{p}$ is written $\{\mathsf{l}\}[\![\mathsf{u}]\!]\ \mathsf{p}$.

[3] Although it strictly subsumes assignment we prefer to keep both.

We introduce the specification of a simple 'toy' protocol as a running example:

$$
\begin{array}{lll}
\Gamma_{\mathsf{Toy}}\ \mathsf{PToy} = \mathsf{labelled\ PToy}\ (\ \mathsf{receive}(\lambda \mathsf{msg'}\ \xi.\ \xi\ (\!|\ \mathsf{msg} := \mathsf{msg'}\ |\!)). & & \{\mathsf{PToy\text{-}:0}\} \\
\qquad [\![\lambda \xi.\ \xi\ (\!|\mathsf{nhip} := \mathsf{ip}\ \xi|\!)]\!] & & \{\mathsf{PToy\text{-}:1}\} \\
\qquad (\quad \langle \mathsf{is\text{-}newpkt} \rangle & & \{\mathsf{PToy\text{-}:2}\} \\
\qquad\quad [\![\lambda \xi.\ \xi\ (\!|\mathsf{no} := \mathsf{max}\ (\mathsf{no}\ \xi)\ (\mathsf{num}\ \xi)|\!)]\!] & & \{\mathsf{PToy\text{-}:3}\} \\
\qquad\quad \mathsf{broadcast}(\lambda \xi.\ \mathsf{pkt}(\mathsf{no}\ \xi,\ \mathsf{ip}\ \xi)).\ \mathsf{Toy}() & & \{\mathsf{PToy\text{-}:4,5}\} \\
\qquad \oplus\ \langle \mathsf{is\text{-}pkt} \rangle & & \{\mathsf{PToy\text{-}:2}\} \\
\qquad\quad (\ \langle \lambda \xi.\ \mathsf{if\ num}\ \xi \geq \mathsf{no}\ \xi\ \mathsf{then}\ \{\xi\}\ \mathsf{else}\ \{\} \rangle & & \{\mathsf{PToy\text{-}:6}\} \\
\qquad\qquad [\![\lambda \xi.\ \xi\ (\!|\mathsf{no} := \mathsf{num}\ \xi|\!)]\!] & & \{\mathsf{PToy\text{-}:7}\} \\
\qquad\qquad [\![\lambda \xi.\ \xi\ (\!|\mathsf{nhip} := \mathsf{sip}\ \xi|\!)]\!] & & \{\mathsf{PToy\text{-}:8}\} \\
\qquad\qquad \mathsf{broadcast}(\lambda \xi.\ \mathsf{pkt}(\mathsf{no}\ \xi,\ \mathsf{ip}\ \xi)).\ \mathsf{Toy}() & & \{\mathsf{PToy\text{-}:9,10}\} \\
\qquad\quad \oplus\ \langle \lambda \xi.\ \mathsf{if\ num}\ \xi < \mathsf{no}\ \xi\ \mathsf{then}\ \{\xi\}\ \mathsf{else}\ \{\} \rangle & & \{\mathsf{PToy\text{-}:6}\} \\
\qquad\quad \mathsf{Toy}()))\ , & & \{\mathsf{PToy\text{-}:11}\}
\end{array}
$$

where PToy is the process name, is-newpkt and is-pkt are guards that unpack the contents of msg, and Toy() is an abbreviation that clears some variables before a call(PToy). The function labelled associates its argument PToy paired with a number to every prefix constructor. There are two types of messages: newpkt (data, dst), from which is-newpkt copies data to the variable num, and pkt (data, src), from which is-pkt copies data into num and src into sip.

The corresponding sequential model is an automaton—a record[4] of two fields: a set of initial states and a set of transitions—parameterized by an address i:

$$\mathsf{ptoy\ i} = (\!|\mathsf{init} = \{(\mathsf{toy\text{-}init\ i},\ \Gamma_{\mathsf{Toy}}\ \mathsf{PToy})\},\ \mathsf{trans} = \mathsf{seqp\text{-}sos}\ \Gamma_{\mathsf{Toy}}|\!)\ ,$$

where toy-init i yields the initial data state $(\!|\mathsf{ip} = \mathsf{i},\ \mathsf{no} = 0,\ \mathsf{nhip} = \mathsf{i},\ \mathsf{msg} = \mathsf{SOME\ x.\ True},\ \mathsf{num} = \mathsf{SOME\ x.\ True},\ \mathsf{sip} = \mathsf{SOME\ x.\ True}|\!)$. The last three variables are initialized to arbitrary values, as they are considered local—they are explicitly reinitialized before each call(PToy). This is the biggest departure from the original definition of AWN; it simplifies the treatment of call, as we show in Section 3.1, and facilitates working with automata where variable locality makes little sense.

2.2 Local Parallel Composition

Message sending protocols must nearly always be input-enabled, that is, nodes should always be in a state where they can receive messages. To achieve this, and to model asynchronous message transmission, the protocol process is combined with a queue model, qmsg, that continually appends received messages onto an

$$\frac{(\mathsf{s,\ a,\ s'}) \in \mathsf{S} \qquad \bigwedge \mathsf{m.\ a} \neq \mathsf{receive\ m}}{((\mathsf{s,\ t}),\ \mathsf{a},\ (\mathsf{s',\ t})) \in \mathsf{parp\text{-}sos\ S\ T}} \qquad \frac{(\mathsf{t,\ a,\ t'}) \in \mathsf{T} \qquad \bigwedge \mathsf{m.\ a} \neq \mathsf{send\ m}}{((\mathsf{s,\ t}),\ \mathsf{a},\ (\mathsf{s,\ t'})) \in \mathsf{parp\text{-}sos\ S\ T}}$$

$$\frac{(\mathsf{s,\ receive\ m,\ s'}) \in \mathsf{S} \qquad (\mathsf{t,\ send\ m,\ t'}) \in \mathsf{T}}{((\mathsf{s,\ t}),\ \tau,\ (\mathsf{s',\ t'})) \in \mathsf{parp\text{-}sos\ S\ T}}$$

Fig. 2. SOS rules for parallel processes: parp-sos

[4] The generic record has type ('s, 'a) automaton, where the type 's is the domain of states, here pairs of data records and control terms, and 'a is the domain of actions.

$$\frac{(\mathsf{s},\ \mathsf{groupcast\ D\ m},\ \mathsf{s'}) \in \mathsf{S}}{(\mathsf{s}^{\mathsf{i}}_{\mathsf{R}},\ (\mathsf{R} \cap \mathsf{D}){:}{*}\mathsf{cast}(\mathsf{m}),\ \mathsf{s'}^{\mathsf{i}}_{\mathsf{R}}) \in \mathsf{node\text{-}sos\ S}} \qquad \frac{(\mathsf{s},\ \mathsf{receive\ m},\ \mathsf{s'}) \in \mathsf{S}}{(\mathsf{s}^{\mathsf{i}}_{\mathsf{R}},\ \{\mathsf{i}\}\neg\emptyset{:}\mathsf{arrive}(\mathsf{m}),\ \mathsf{s'}^{\mathsf{i}}_{\mathsf{R}}) \in \mathsf{node\text{-}sos\ S}}$$

$$(\mathsf{s}^{\mathsf{i}}_{\mathsf{R}},\ \emptyset\neg\{\mathsf{i}\}{:}\mathsf{arrive}(\mathsf{m}),\ \mathsf{s}^{\mathsf{i}}_{\mathsf{R}}) \in \mathsf{node\text{-}sos\ S} \qquad (\mathsf{s}^{\mathsf{i}}_{\mathsf{R}},\ \mathsf{connect}(\mathsf{i},\ \mathsf{i'}),\ \mathsf{s}^{\mathsf{i}}_{\mathsf{R}\,\cup\,\{\mathsf{i'}\}}) \in \mathsf{node\text{-}sos\ S}$$

Fig. 3. SOS rules for nodes: examples from node-sos

internal list and offers to send the head message to the protocol process: ptoy i $\langle\langle$ qmsg. The *local parallel* operator is a function over automata:

$$\mathsf{s} \langle\langle \mathsf{t} = (\!|\mathsf{init} = \mathsf{init\ s} \times \mathsf{init\ t},\ \mathsf{trans} = \mathsf{parp\text{-}sos\ (trans\ s)\ (trans\ t)}|\!)\,.$$

The rules for parp-sos are shown in Figure 2.

2.3 Nodes

At the node level, a local process np is wrapped in a layer that records its address i and tracks the set of neighbouring node addresses, initially R_i:

$$\langle \mathsf{i} : \mathsf{np} : \mathsf{R}_i \rangle = (\!|\mathsf{init} = \{\mathsf{s}^{\mathsf{i}}_{\mathsf{R}_i} \mid \mathsf{s} \in \mathsf{init\ np}\},\ \mathsf{trans} = \mathsf{node\text{-}sos\ (trans\ np)}|\!)\,.$$

Node states are denoted $\mathsf{s}^{\mathsf{i}}_{\mathsf{R}}$. Figure 3 presents rules typical of node-sos. Output network synchronizations, like groupcast, are filtered by the list of neighbours to become *cast actions. The H¬K:arrive(m) action—in Figure 3 instantiated as $\emptyset\neg\{\mathsf{i}\}$:arrive(m), and $\{\mathsf{i}\}\neg\emptyset$:arrive(m)—is used to model a message m received by nodes in H and not by those in K. The connect(i, i') adds node i' to the set of neighbours of node i; disconnect(i, i') works similarly.

2.4 Partial Networks

Partial networks are specified as values of type net-tree, that is, as a node $\langle \mathsf{i};\ \mathsf{R}_i \rangle$ with address i and a set of initial neighbours R_i, or a composition of two net-trees $\mathsf{p}_1 \parallel \mathsf{p}_2$. The function pnet maps such a value, together with the process np i to execute at each node i, here parameterized by an address, to an automaton:

$$\begin{aligned}
&\mathsf{pnet\ np}\ \langle \mathsf{i};\ \mathsf{R}_i \rangle \quad = \langle \mathsf{i} : \mathsf{np\ i} : \mathsf{R}_i \rangle \\
&\mathsf{pnet\ np}\ (\mathsf{p}_1 \parallel \mathsf{p}_2) = (\!|\mathsf{init} = \{\mathsf{s}_1 \mathbin{||} \mathsf{s}_2 \mid \mathsf{s}_1 \in \mathsf{init\ (pnet\ np\ p_1)} \wedge \mathsf{s}_2 \in \mathsf{init\ (pnet\ np\ p_2)}\}, \\
&\qquad\qquad\qquad \mathsf{trans} = \mathsf{pnet\text{-}sos\ (trans\ (pnet\ np\ p_1))\ (trans\ (pnet\ np\ p_2))}|\!)\,,
\end{aligned}$$

The states of such automata mirror the tree structure of the network term; we denote composed states $\mathsf{s}_1 \mathbin{||} \mathsf{s}_2$. This structure, and the node addresses, remain constant during an execution. These definitions suffice to model an example three node network of toy processes:

$$\mathsf{pnet}\ (\lambda\mathsf{i}.\ \mathsf{ptoy\ i}\ \langle\langle\ \mathsf{qmsg})\ (\langle \mathsf{A};\ \{\mathsf{B}\} \rangle \parallel \langle \mathsf{B};\ \{\mathsf{A},\ \mathsf{C}\} \rangle \parallel \langle \mathsf{C};\ \{\mathsf{B}\} \rangle)\,.$$

Figure 4 presents rules typical of pnet-sos. There are rules where only one node acts, like the one shown for τ, and rules where all nodes act, like those for *cast and arrive. The latter ensure—since qmsg is always ready to receive m—that a partial network can always perform an H¬K:arrive(m) for any combination of H and K consistent with its node addresses, but that pairing with an R:*cast(m) restricts the possibilities to the one consistent with the destinations in R.

$$\frac{(\mathsf{s},\ \mathsf{R{:}{*}cast(m)},\ \mathsf{s'}) \in \mathsf{S} \quad (\mathsf{t},\ \mathsf{H}\neg\mathsf{K{:}arrive(m)},\ \mathsf{t'}) \in \mathsf{T} \quad \mathsf{H} \subseteq \mathsf{R} \quad \mathsf{K} \cap \mathsf{R} = \emptyset}{(\mathsf{s} \parallel \mathsf{t},\ \mathsf{R{:}{*}cast(m)},\ \mathsf{s'} \parallel \mathsf{t'}) \in \mathsf{pnet\text{-}sos\ S\ T}}$$

$$\frac{(\mathsf{s},\ \mathsf{H}\neg\mathsf{K{:}arrive(m)},\ \mathsf{s'}) \in \mathsf{S} \quad (\mathsf{t},\ \mathsf{H'}\neg\mathsf{K'{:}arrive(m)},\ \mathsf{t'}) \in \mathsf{T}}{(\mathsf{s} \parallel \mathsf{t},\ (\mathsf{H} \cup \mathsf{H'})\neg(\mathsf{K} \cup \mathsf{K'}){:}\mathsf{arrive(m)},\ \mathsf{s'} \parallel \mathsf{t'}) \in \mathsf{pnet\text{-}sos\ S\ T}} \qquad \frac{(\mathsf{s},\ \tau,\ \mathsf{s'}) \in \mathsf{S}}{(\mathsf{s} \parallel \mathsf{t},\ \tau,\ \mathsf{s'} \parallel \mathsf{t}) \in \mathsf{pnet\text{-}sos\ S\ T}}$$

Fig. 4. SOS rules for partial networks: examples from pnet-sos

2.5 Complete Networks

The last layer closes a network to further interactions with an environment; the *cast action becomes a τ and H¬K:arrive(m) is forbidden:

$$\mathsf{closed\ A} = \mathsf{A}(\!|\mathsf{trans} := \mathsf{cnet\text{-}sos\ (trans\ A)}|\!)\,.$$

The rules for cnet-sos are straight-forward and not presented here.

3 Basic Invariance

This paper only considers proofs of invariance, that is, properties of reachable states. The basic definitions are classic [14, Part III].

Definition 1 (reachability). *Given an automaton A and an assumption I over actions, reachable A I is the smallest set defined by the rules:*

$$\frac{s \in \mathit{init}\ A}{s \in \mathit{reachable}\ A\ I} \qquad \frac{s \in \mathit{reachable}\ A\ I \quad (s,\ a,\ s') \in \mathit{trans}\ A \quad I\ a}{s' \in \mathit{reachable}\ A\ I}$$

Definition 2 (invariance). *Given an automaton A and an assumption I, a predicate P is* invariant*, denoted* $A \models (I \rightarrow)\ P$*, iff* $\forall\, s \in \mathit{reachable}\ A\ I.\ P\ s$.

We state reachability relative to an assumption on (input) actions I. When I is λ-. True, we write simply A ⊨ P.

Definition 3 (step invariance). *Given an automaton A and an assumption I, a predicate P is* step invariant*, denoted* $A \models (I \rightarrow)\ P$*, iff*

$$\forall\, a.\ I\ a \longrightarrow (\forall\, s \in \mathit{reachable}\ A\ I.\ \forall\, s'.\ (s,\ a,\ s') \in \mathit{trans}\ A \longrightarrow P\ (s,\ a,\ s'))\,.$$

Our invariance proofs follow the compositional strategy recommended in [18, §1.6.2]. That is, we show properties of sequential process automata using the induction principle of Definition 1, and then apply generic proof rules to successively lift such properties over each of the other layers. The inductive assertion method, as stated in rule INV-B of [13], requires a finite set of transition schemas, which, together with the obligation on initial states yields a set of sufficient verification conditions. We develop this set in Section 3.1 and use it to derive the main proof rule presented in Section 3.2 together with some examples.

3.1 Control Terms

Given a specification Γ over finitely many process names, we can generate a finite set of verification conditions because transitions from ('s, 'p, 'l) seqp terms always yield subterms of terms in Γ. But, rather than simply consider the set of all subterms, we prefer to define a subset of 'control terms' that reduces the number of verification conditions, avoids tedious duplication in proofs, and corresponds with the obligations considered in pen-and-paper proofs. The main idea is that the $\oplus$ and call operators serve only to combine process terms: they are, in a sense, executed recursively by seqp-sos to determine the actions that a term offers to its environment. This is made precise by defining a relation between sequential process terms.

Definition 4 ($\rightsquigarrow_\Gamma$). *For a (recursive) specification Γ, let $\rightsquigarrow_\Gamma$ be the smallest relation such that $(p_1 \oplus p_2) \rightsquigarrow_\Gamma p_1$, $(p_1 \oplus p_2) \rightsquigarrow_\Gamma p_2$, and $(\mathsf{call}(pn)) \rightsquigarrow_\Gamma \Gamma\ pn$.*

We write $\rightsquigarrow_\Gamma{}^*$ for its reflexive transitive closure. We consider a specification to be *well formed*, when the inverse of this relation is well founded:

$$\mathsf{wellformed}\ \Gamma = \mathsf{wf}\ \{(\mathsf{q}, \mathsf{p}) \mid \mathsf{p} \rightsquigarrow_\Gamma \mathsf{q}\}.$$

Most of our lemmas only apply to well formed specifications, since otherwise functions over the terms they contain cannot be guaranteed to terminate. Neither of these two specifications is well formed: $\Gamma_a(1) = \mathsf{p} \oplus \mathsf{call}(1)$; $\Gamma_b(\mathsf{n}) = \mathsf{call}(\mathsf{n} + 1)$.

We will also need a set of 'start terms'—the subterms that can act directly.

Definition 5 (sterms). *Given a wellformed Γ and a sequential process term p, sterms Γ p is the set of maximal elements related to p by the reflexive transitive closure of the $\rightsquigarrow_\Gamma$ relation*[5]*:*

$$\begin{aligned}
&\mathsf{sterms}\ \Gamma\ (p_1 \oplus p_2) &&= \mathsf{sterms}\ \Gamma\ p_1 \cup \mathsf{sterms}\ \Gamma\ p_2,\\
&\mathsf{sterms}\ \Gamma\ (\mathsf{call}(pn)) &&= \mathsf{sterms}\ \Gamma\ (\Gamma\ pn), \text{ and,}\\
&\mathsf{sterms}\ \Gamma\ p &&= \{p\} \text{ otherwise.}
\end{aligned}$$

We also define 'local start terms' by $\mathsf{stermsl}\ (\mathsf{p}_1 \oplus \mathsf{p}_2) = \mathsf{stermsl}\ \mathsf{p}_1 \cup \mathsf{stermsl}\ \mathsf{p}_2$ and otherwise $\mathsf{stermsl}\ \mathsf{p} = \{\mathsf{p}\}$ to permit the sufficient syntactic condition that a specification Γ is well formed if $\mathsf{call}(\mathsf{pn'}) \notin \mathsf{stermsl}\ (\Gamma\ \mathsf{pn})$.

Similarly to the way that start terms act as direct sources of transitions, we define 'derivative terms' giving possible active destinations of transitions.

Definition 6 (dterms). *Given a wellformed Γ and a sequential process term p, dterms p is defined by:*

$$\begin{aligned}
&\mathsf{dterms}\ \Gamma\ (p_1 \oplus p_2) &&= \mathsf{dterms}\ \Gamma\ p_1 \cup \mathsf{dterms}\ \Gamma\ p_2,\\
&\mathsf{dterms}\ \Gamma\ (\mathsf{call}(pn)) &&= \mathsf{dterms}\ \Gamma\ (\Gamma\ pn),\\
&\mathsf{dterms}\ \Gamma\ (\{l\}[\![u]\!]\ p) &&= \mathsf{sterms}\ \Gamma\ p,\\
&\mathsf{dterms}\ \Gamma\ (\{l\}\mathsf{unicast}(s_{ip}, s_{msg})\ .\ p \triangleright q) &&= \mathsf{sterms}\ \Gamma\ p \cup \mathsf{sterms}\ \Gamma\ q, \text{ and so on.}
\end{aligned}$$

[5] This characterization is equivalent to $\{\mathsf{q} \mid \mathsf{p} \rightsquigarrow_\Gamma{}^* \mathsf{q} \wedge (\nexists \mathsf{q'}.\ \mathsf{q} \rightsquigarrow_\Gamma \mathsf{q'})\}$. Termination follows from wellformed Γ, that is, $\mathsf{wellformed}\ \Gamma \Longrightarrow \mathsf{sterms\text{-}dom}\ (\Gamma, \mathsf{p})$ for all p.

These derivative terms overapproximate the set of reachable sterms, since they do not consider the truth of guards nor the willingness of communication partners.

These auxiliary definitions lead to a succinct definition of the set of control terms of a specification.

Definition 7 (cterms). *For a specification Γ, cterms is the smallest set where:*

$$\frac{p \in \mathit{sterms}\ \Gamma\ (\Gamma\ \mathit{pn})}{p \in \mathit{cterms}\ \Gamma} \qquad \frac{pp \in \mathit{cterms}\ \Gamma \quad p \in \mathit{dterms}\ \Gamma\ pp}{p \in \mathit{cterms}\ \Gamma}$$

It is also useful to define a local version independent of any specification.

Definition 8 (ctermsl). *Let ctermsl be the smallest set defined by:*

$\mathit{ctermsl}\ (p_1 \oplus p_2) = \mathit{ctermsl}\ p_1 \cup \mathit{ctermsl}\ p_2$,
$\mathit{ctermsl}\ (\mathit{call}(pn)) = \{\mathit{call}(pn)\}$,
$\mathit{ctermsl}\ (\{l\}[\![u]\!]\ p) = \{\{l\}[\![u]\!]\ p\} \cup \mathit{ctermsl}\ p$, *and so on.*

Including call terms ensures that $\mathsf{q} \in \mathsf{stermsl\ p}$ implies $\mathsf{q} \in \mathsf{ctermsl\ p}$, which facilitates proofs. For wellformed Γ, ctermsl allows an alternative definition of cterms,

$$\mathsf{cterms}\ \Gamma = \{\mathsf{p} \mid \exists\, \mathsf{pn.}\ \mathsf{p} \in \mathsf{ctermsl}\ (\Gamma\ \mathsf{pn}) \wedge \mathsf{not\text{-}call\ p}\}. \tag{1}$$

While the original definition is convenient for developing the meta-theory, due to the accompanying induction principle, this one is more useful for systematically generating the set of control terms of a specification, and thus, we will see, sets of verification conditions. And, for wellformed Γ, we have as a corollary

$$\mathsf{cterms}\ \Gamma = \{\mathsf{p} \mid \exists\, \mathsf{pn.}\ \mathsf{p} \in \mathsf{subterms}\ (\Gamma\ \mathsf{pn}) \wedge \mathsf{not\text{-}call\ p} \wedge \mathsf{not\text{-}choice\ p}\}, \tag{2}$$

where subterms, not-call, and not-choice are defined in the obvious way.

We show that cterms over-approximates the set of reachable control states.

Lemma 1. *For wellformed Γ and automaton A where control-within Γ (init A) and trans A = seqp-sos Γ, if $(\xi, p) \in$ reachable A I and $q \in$ sterms Γ p then $q \in$ cterms Γ.*

The predicate $\mathsf{control\text{-}within}\ \Gamma\ \sigma = \forall\, (\xi, \mathsf{p}) \in \sigma.\ \exists\, \mathsf{pn.}\ \mathsf{p} \in \mathsf{subterms}\ (\Gamma\ \mathsf{pn})$ serves to state that the initial control state is within the specification.

3.2 Basic Proof Rule and Invariants

Using the definition of invariance (Definition 2), we can state a basic property of an instance of the toy process:

$$\mathsf{ptoy\ i} \models \mathsf{onl}\ \Gamma_{\mathsf{Toy}}\ (\lambda(\xi, \mathsf{l}).\ \mathsf{l} \in \{\mathsf{PToy\text{-}:2..PToy\text{-}:8}\} \longrightarrow \mathsf{nhip}\ \xi = \mathsf{ip}\ \xi), \tag{3}$$

This invariant states that between the lines labelled PToy-:2 and PToy-:8, that is, after the assignment of PToy-:1 until before the assignment of PToy-:8, the values of nhip and ip are equal; onl Γ P, defined as $\lambda(\xi, \mathsf{p}).\ \forall\, \mathsf{l} \in \mathsf{labels}\ \Gamma\ \mathsf{p.}\ \mathsf{P}\ (\xi, \mathsf{l})$, extracts labels from control states.[6] Invariants like these are solved using a procedure whose soundness is justified as a theorem. The proof exploits (1) and Lemma 1.

[6] Using labels in this way is standard, see, for instance, [13, Chap. 1], or the 'assertion networks' of [18, §2.5.1]. Isabelle rapidly dispatches all the uninteresting cases.

Theorem 1. *To prove* $A \models (I \rightarrow)$ *onl* Γ P, *where* wellformed Γ, simple-labels Γ, control-within Γ (init A), *and* trans A = seqp-sos Γ, *it suffices*

(init) *for arbitrary* $(\xi, p) \in$ init A *and* $l \in$ labels Γ p, *to show* P (ξ, l), *and,*

(step) *for arbitrary* $p \in$ ctermsl $(\Gamma$ pn$)$, *but* not-call p, *and* $l \in$ labels Γ p, *given that* $p \in$ sterms Γ pp *for some* $(\xi, pp) \in$ reachable A I, *to assume* P (ξ, l) *and* I a, *and then for any* (ξ', q) *such that* $((\xi, p), a, (\xi', q)) \in$ seqp-sos Γ *and* $l' \in$ labels Γ q, *to show* P (ξ', l').

Here, $\mathsf{simple\text{-}labels}\ \Gamma = \forall \mathsf{pn}.\ \forall \mathsf{p} \in \mathsf{subterms}\ (\Gamma\ \mathsf{pn}).\ \exists!\, \mathsf{l}.\ \mathsf{labels}\ \Gamma\ \mathsf{p} = \{\mathsf{l}\}$: each control term must have exactly one label, that is, $\oplus$ terms must be labelled consistently.

We incorporate this theorem into a tactic that (1) applies the introduction rule, (2) replaces $\mathsf{p} \in \mathsf{ctermsl}\ (\Gamma\ \mathsf{pn})$ by a disjunction over the values of pn, (3) applies Definition 8 and repeated simplifications of Γs and eliminations on disjunctions to generate one subgoal (verification condition) for each control term, (4) replaces control term derivatives, the subterms in Definition 6, by fresh variables, and, finally, (5) tries to solve each subgoal by simplification. Step 4 replaces potentially large control terms by their (labelled) heads, which is important for readability and prover performance. The tactic takes as arguments a list of existing invariants to include after having applied the introduction rule and a list of lemmas for trying to solve any subgoals that survive the final simplification. There are no schematic variables in the subgoals and we benefit greatly from Isabelle's PARALLEL_GOALS tactical [22].

In practice, one states an invariant, applies the tactic, and examines the resulting goals. One may need new lemmas for functions over the data state or explicit proofs for difficult goals. That said, the tactic generally dispatches the uninteresting goals, and the remaining ones typically correspond with the cases treated explicitly in manual proofs [4].

For step invariants, we show a counterpart to Theorem 1, and declare it to the tactic. Then we can show, for our example, that the value of no never decreases:

$$\mathsf{ptoy\ i} \models (\lambda((\xi, \text{-}), \text{-}, (\xi', \text{-})).\ \mathsf{no}\ \xi \leq \mathsf{no}\ \xi').$$

4 Open Invariance

The analysis of network protocols often requires 'inter-node' invariants, like

$$\mathsf{wf\text{-}net\text{-}tree\ n} \Longrightarrow \mathsf{closed}\ (\mathsf{pnet}\ (\lambda \mathsf{i}.\ \mathsf{ptoy\ i}\ \langle\langle\ \mathsf{qmsg})\ \mathsf{n}) \models \mathsf{netglobal}\ (\lambda\sigma.\ \forall \mathsf{i}.\ \mathsf{no}\ (\sigma\ \mathsf{i}) \leq \mathsf{no}\ (\sigma\ (\mathsf{nhip}\ (\sigma\ \mathsf{i})))), \quad (4)$$

which states that, for any net-tree with disjoint node addresses (wf-net-tree n), the value of no at a node is never greater than its value at the 'next hop'—the address in nhip. This is a property of a global state σ mapping addresses to corresponding data states. Such a global state is readily constructed with:

$$\mathsf{netglobal\ P} = \lambda \mathsf{s}.\ \mathsf{P}\ (\mathsf{default\ toy\text{-}init}\ (\mathsf{netlift\ fst\ s})),$$
$$\mathsf{default\ df\ f} = (\lambda \mathsf{i}.\ \mathsf{case\ f\ i\ of\ None} \Rightarrow \mathsf{df\ i} \mid \mathsf{Some\ s} \Rightarrow \mathsf{s}), \text{ and}$$

$$\begin{aligned} \mathsf{netlift\ sr}\ (\mathsf{s}^{\mathsf{i}}_{\mathsf{R}}) &= [\mathsf{i} \mapsto \mathsf{fst\ (sr\ s)}] \\ \mathsf{netlift\ sr}\ (\mathsf{s} \parallel \mathsf{t}) &= \mathsf{netlift\ sr\ s} +\!\!+ \mathsf{netlift\ sr\ t}. \end{aligned}$$

The applications of fst elide the state of qmsg and the protocol's control state.[7]

While we can readily state inter-node invariants of a complete model, showing them compositionally is another issue. Sections 4.1 and 4.2 present a way to state and prove such invariants at the level of sequential processes—that is, with only ptoy i left of the turnstile. Sections 4.3 and 4.4 present, respectively, rules for lifting such results to network models and for recovering invariants like (4).

4.1 The Open Model

Rather than instantiate the 's of ('s, 'p, 'l) seqp with elements ξ of type state, our solution introduces a global state σ of type ip $\Rightarrow$ state. This necessitates a stack of new SOS rules that we call the *open model*; Figure 5 shows some representatives.

The rules of oseqp-sos are parameterized by an address i and constrain only that entry of the global state, either to say how it changes (σ' i = u (σ i)) or that it does not (σ' i = σ i). The rules for oparp-sos only allow the first sub-process to constrain σ. This choice is disputable: it precludes comparing the states of qmsgs (and any other local filters) across a network, but is also simplifies the mechanics and use of this layer of the framework.[8] The sets onode-sos and opnet-sos need not be parameterized since they are generated inductively from lower layers. Together they constrain subsets of elements of σ. This occurs naturally for rules like those for arrive and *cast, where the synchronous communication serves as a conjunction of constraints on sub-ranges of σ. But for others that normally only constrain a single element, like those for τ, assumptions ($\forall$ j $\neq$ i. σ' j = σ j) are introduced here and later dispatched (Section 4.4). The rules for ocnet-sos, not shown, are similar—elements not addressed within a model may not change.

The stack of operators and model layers described in Section 2 is refashioned to use the new transition rules and to distinguish the global state, which is preserved as the fst element across layers, from the local state elements which are combined in the snd element as before.

For instance, a sequential instance of the toy protocol is defined as

$$\mathsf{optoy\ i} = (\!|\mathsf{init} = \{(\mathsf{toy\text{-}init},\ \Gamma_{\mathsf{Toy}}\ \mathsf{PToy})\},\ \mathsf{trans} = \mathsf{oseqp\text{-}sos}\ \Gamma_{\mathsf{Toy}}\ \mathsf{i}|\!)\,,$$

combined with the standard qmsg process using the operator

$$\begin{aligned}\mathsf{s}\ \langle\langle_i\ \mathsf{t} = (\!|&\mathsf{init} = \{(\sigma,\ (\mathsf{s}_l,\ \mathsf{t}_l))\ |\ (\sigma,\ \mathsf{s}_l) \in \mathsf{init\ s} \wedge \mathsf{t}_l \in \mathsf{init\ t}\},\\ &\mathsf{trans} = \mathsf{oparp\text{-}sos\ i\ (trans\ s)\ (trans\ t)}|\!)\ ,\end{aligned}$$

and lifted to the node level via the open node constructor

$$\langle \mathsf{i} : \mathsf{onp} : \mathsf{R}_i \rangle_o = (\!|\mathsf{init} = \{(\sigma,\ \mathsf{s}^{\mathsf{i}}_{\mathsf{R}_i})\ |\ (\sigma,\ \mathsf{s}) \in \mathsf{init\ onp}\},\ \mathsf{trans} = \mathsf{onode\text{-}sos\ (trans\ onp)}|\!)\,.$$

Similarly, to map a net-tree term to an open model we define:

$$\begin{aligned}\mathsf{opnet\ onp}\ \langle \mathsf{i};\ \mathsf{R}_i \rangle &= \langle \mathsf{i} : \mathsf{onp\ i} : \mathsf{R}_i \rangle_o\\ \mathsf{opnet\ onp}\ (\mathsf{p}_1 \parallel \mathsf{p}_2) &= (\!|\mathsf{init} = \{(\sigma,\ \mathsf{s}_1 \uplus \mathsf{s}_2)\ |\ (\sigma,\ \mathsf{s}_1) \in \mathsf{init\ (opnet\ onp\ p_1)}\\ &\qquad \wedge\ (\sigma,\ \mathsf{s}_2) \in \mathsf{init\ (opnet\ onp\ p_2)}\\ &\qquad \wedge\ \mathsf{net\text{-}ips\ s_1} \cap \mathsf{net\text{-}ips\ s_2} = \emptyset\},\\ &\qquad \mathsf{trans} = \mathsf{opnet\text{-}sos\ (trans(opnet\ onp\ p_1))\ (trans(opnet\ onp\ p_2))}|\!)\,.\end{aligned}$$

[7] The formulation here is a technical detail: sr corresponds to netlift as np does to pnet.

[8] The treatment of the other layers is completely independent of this choice.

$$\frac{\sigma'\ \mathsf{i} = \mathsf{u}\ (\sigma\ \mathsf{i})}{((\sigma,\ \{\mathsf{I}\}[\![\mathsf{u}]\!]\ \mathsf{p}),\ \tau,\ (\sigma',\ \mathsf{p})) \in \mathsf{oseqp\text{-}sos}\ \Gamma\ \mathsf{i}} \qquad \frac{((\sigma,\ \mathsf{p}),\ \mathsf{a},\ (\sigma',\ \mathsf{p}')) \in \mathsf{oseqp\text{-}sos}\ \Gamma\ \mathsf{i}}{((\sigma,\ \mathsf{p} \oplus \mathsf{q}),\ \mathsf{a},\ (\sigma',\ \mathsf{p}')) \in \mathsf{oseqp\text{-}sos}\ \Gamma\ \mathsf{i}}$$

$$\frac{\sigma'\ \mathsf{i} = \sigma\ \mathsf{i}}{((\sigma,\ \{\mathsf{I}\}\mathsf{unicast}(\mathsf{s}_{ip}, \mathsf{s}_{msg})\ .\ \mathsf{p} \triangleright \mathsf{q}),\ \mathsf{unicast}\ (\mathsf{s}_{ip}\ (\sigma\ \mathsf{i}))\ (\mathsf{s}_{msg}\ (\sigma\ \mathsf{i})),\ (\sigma',\mathsf{p})) \in \mathsf{oseqp\text{-}sos}\ \Gamma\ \mathsf{i}}$$

(a) Sequential processes: examples from oseqp-sos.

$$\frac{((\sigma,\ \mathsf{s}),\ \mathsf{receive\ m},\ (\sigma',\ \mathsf{s}')) \in \mathsf{S} \qquad (\mathsf{t},\ \mathsf{send\ m},\ \mathsf{t}') \in \mathsf{T}}{((\sigma,\ (\mathsf{s},\ \mathsf{t})),\ \tau,\ (\sigma',\ (\mathsf{s}',\ \mathsf{t}'))) \in \mathsf{oparp\text{-}sos\ i\ S\ T}}$$

(b) Parallel processes: example from oparp-sos.

$$\frac{((\sigma,\mathsf{s}),\ \mathsf{receive\ m},\ (\sigma',\mathsf{s}')) \in \mathsf{S}}{((\sigma,\mathsf{s}^{\mathsf{i}}_{\mathsf{R}}),\ \{\mathsf{i}\}\neg\emptyset\mathsf{:arrive(m)},\ (\sigma',\mathsf{s}'^{\mathsf{i}}_{\mathsf{R}})) \in \mathsf{onode\text{-}sos\ S}} \qquad \frac{((\sigma,\mathsf{s}),\tau,\ (\sigma',\mathsf{s}')) \in \mathsf{S} \quad \forall\, \mathsf{j} \neq \mathsf{i}.\ \sigma'\,\mathsf{j} = \sigma\,\mathsf{j}}{((\sigma,\mathsf{s}^{\mathsf{i}}_{\mathsf{R}}),\tau,\ (\sigma',\mathsf{s}'^{\mathsf{i}}_{\mathsf{R}})) \in \mathsf{onode\text{-}sos\ S}}$$

(c) Nodes: examples from onode-sos.

$$\frac{((\sigma,\ \mathsf{s}),\ \mathsf{H}\neg\mathsf{K:arrive(m)},\ (\sigma',\ \mathsf{s}')) \in \mathsf{S} \qquad ((\sigma,\ \mathsf{t}),\ \mathsf{H}'\neg\mathsf{K}'\mathsf{:arrive(m)},\ (\sigma',\ \mathsf{t}')) \in \mathsf{T}}{((\sigma,\ \mathsf{s} \parallel \mathsf{t}),\ (\mathsf{H} \cup \mathsf{H}')\neg(\mathsf{K} \cup \mathsf{K}')\mathsf{:arrive(m)},\ (\sigma',\ \mathsf{s}' \parallel \mathsf{t}')) \in \mathsf{opnet\text{-}sos\ S\ T}}$$

(d) Partial networks: example from opnet-sos.

Fig. 5. SOS rules for the open model (cf. Figures 1, 2, 3, and 4)

This definition is non-empty only for well-formed net-trees (net-ips gives the set of node addresses in the state of a partial network). Including such a constraint within the open model, rather than as a separate assumption like the wf-net-tree n in (4), eliminates an annoying technicality from the inductions described in Section 4.3. As with the extra premises in the open SOS rules, we can freely adjust the open model to facilitate proofs but each 'encoded assumption' becomes an obligation to be discharged in the transfer lemma of Section 4.4.

An operator for adding the last layer is also readily defined by

$$\mathsf{oclosed\ A} = \mathsf{A}(\!|\mathsf{trans} := \mathsf{ocnet\text{-}sos\ (trans\ A)}|\!)\,,$$

giving all the definitions necessary to turn a standard model into an open one.

4.2 Open Invariants

The basic definitions of reachability and invariance, Definitions 1–3, apply to open models, but constructing a compositional proof requires considering the effects of both synchronized and interleaved actions of possible environments.

Definition 9 (open reachability). *Given an automaton A and assumptions S and U over, respectively, synchronized and interleaved actions,* oreachable *A S U is the smallest set defined by the rules:*

$$\frac{(\sigma,\ p) \in \mathit{init\ A}}{(\sigma,\ p) \in \mathit{oreachable\ A\ S\ U}} \qquad \frac{(\sigma,\ p) \in \mathit{oreachable\ A\ S\ U} \quad U\ \sigma\ \sigma'}{(\sigma',\ p) \in \mathit{oreachable\ A\ S\ U}}$$

$$\frac{(\sigma,\ p) \in \mathit{oreachable\ A\ S\ U} \quad ((\sigma,\ p),\ a,\ (\sigma',\ p')) \in \mathit{trans\ A} \quad S\ \sigma\ \sigma'\ a}{(\sigma',\ p') \in \mathit{oreachable\ A\ S\ U}}$$

In practice, we use restricted forms of the assumptions S and U, respectively,

$$\text{otherwith E N I } \sigma\ \sigma' \text{ a} = (\forall \text{i. i} \notin \text{N} \longrightarrow \text{E } (\sigma \text{ i}) (\sigma' \text{ i})) \wedge \text{I } \sigma \text{ a}, \tag{5}$$

$$\text{other F N } \sigma\ \sigma' = \forall \text{i. if i} \in \text{N then } \sigma' \text{ i} = \sigma \text{ i else F } (\sigma \text{ i}) (\sigma' \text{ i}). \tag{6}$$

The former permits the restriction of possible environments (E) and also the extraction of information from shared actions (I). The latter restricts (F) the effects of interleaved actions, which may only change non-local state elements.

Definition 10 (open invariance). *Given an automaton A and assumptions S and U over, respectively, synchronized and interleaved actions, a predicate P is an* open invariant*, denoted* $A \models (S, U \rightarrow) P$*, iff* $\forall s \in \text{oreachable } A\ S\ U.\ P\ s$.

It follows easily that existing invariants can be made open: most invariants can be shown in the basic context but still exploited in the more complicated one.

Lemma 2. *Given an invariant* $A \models (I \rightarrow) P$ *where* $\text{trans } A = \text{seqp-sos } \Gamma$*, and any F, there is an open invariant* $A' \models (\lambda\text{- -. } I, \text{other } F \{i\} \rightarrow) (\lambda(\sigma, p).\ P\ (\sigma\ i, p))$ *where* $\text{trans } A' = \text{oseqp-sos } \Gamma\ i$*, provided that* $\text{init } A = \{(\sigma\ i, p) \mid (\sigma, p) \in \text{init } A'\}$.

Open step invariance and a similar transfer lemma are defined similarly. The meta theory for basic invariants is also readily adapted, in particular,

Theorem 2. *To show* $A \models (S, U \rightarrow) \text{ onl } \Gamma\ P$*, in addition to the conditions and the obligations* ***(init)*** *and* ***(step)*** *of Theorem 1, suitably adjusted, it suffices,*

(env) *for arbitrary* $(\sigma, p) \in \text{oreachable } A\ S\ U$ *and* $l \in \text{labels } \Gamma\ p$*, to assume both* $P\ (\sigma, l)$ *and* $U\ \sigma\ \sigma'$*, and then to show* $P\ (\sigma', l)$.

This theorem is declared to the tactic described in Section 3.2 and proofs proceed as before, but with the new obligation to show invariance over interleaved steps.

We finally have sufficient machinery to state (and prove) Invariant (4) at the level of a sequential process:

$$\begin{aligned}\text{optoy i} \models\ & (\text{otherwith nos-inc } \{\text{i}\}\ (\text{orecvmsg msg-ok}), \text{other nos-inc } \{\text{i}\} \rightarrow) \\ & (\lambda(\sigma, \text{-}).\ \text{no } (\sigma \text{ i}) \leq \text{no } (\sigma\ (\text{nhip } (\sigma \text{ i})))) ,\end{aligned} \tag{7}$$

where $\text{nos-inc } \xi\ \xi' = \text{no } \xi \leq \text{no } \xi'$, orecvmsg applies its given predicate to receive actions and is otherwise true, $\text{msg-ok } \sigma\ (\text{pkt (data, src)}) = (\text{data} \leq \text{no } (\sigma \text{ src}))$, and $\text{msg-ok } \sigma\ (\text{newpkt (data, dst)}) = \text{True}$. So, given that the variables no in the environment never decrease and that incoming pkts reflect the state of the sender, there is a relation between the local node and the next hop. Similar invariants occur in proofs of realistic protocols [4].

4.3 Lifting Open Invariants

The next step is to lift Invariant (7) over each composition operator of the open model. We mostly present the lemmas over oreachable, rather than those for open invariants and step invariants, which follow more or less directly.

The first lifting rule treats composition with the qmsg process. It mixes oreachable and reachable predicates: the former for the automaton being lifted, the latter for properties of qmsg. The properties of qmsg—only received messages are added to the queue and sent messages come from the queue—are shown using the techniques of Section 3.

Lemma 3 (qmsg lifting). *Given* $(\sigma, (s, (q, t))) \in$ *oreachable* $(A \langle\langle_i$ *qmsg*$)$ S U, *where predicates* $S =$ *otherwith* E $\{i\}$ *(orecvmsg* R*) and* $U =$ *other* F $\{i\}$, *and provided (1)* $A \models (S, U \rightarrow) (\lambda((\sigma, \text{-}), \text{-}, (\sigma', \text{-})).\ F\ (\sigma\ i)\ (\sigma'\ i))$, *(2) for all* ξ, ξ', $E\ \xi\ \xi'$ *implies* $F\ \xi\ \xi'$, *(3) for all* σ, σ', m, $\forall j.\ F\ (\sigma\ j)\ (\sigma'\ j)$ *and* $R\ \sigma\ m$ *imply* $R\ \sigma'\ m$, *and, (4)* F *is reflexive, then* $(\sigma, s) \in$ *oreachable* A S U *and* $(q, t) \in$ *reachable qmsg (recvmsg* $(R\ \sigma)$*), and furthermore* $\forall\ m \in$ *set* $q.\ R\ \sigma\ m$.

The key intuition is that every message m received, queued, and sent by qmsg satisfies R σ m. The proof is by induction over oreachable. The R's are preserved when the external environment acts independently (3, 4), when it acts synchronously (2), and when the local process acts (1, 3).

The rule for lifting to the node level adapts assumptions on receive actions (orecvmsg) to arrive actions (oarrivemsg).

Lemma 4 (onode lifting). *If, for all* ξ *and* ξ', $E\ \xi\ \xi'$ *implies* $F\ \xi\ \xi'$, *then given* $(\sigma, s^i_R) \in$ *oreachable* $(\langle i : A : R_i \rangle_o)$ *(otherwith* E $\{i\}$ *(oarrivemsg* I*)) (other* F $\{i\}$*) it follows that* $(\sigma, s) \in$ *oreachable* A *(otherwith* E $\{i\}$ *(orecvmsg* I*)) (other* F $\{i\}$*).*

The sole condition is needed because certain node-level actions—namely connect, disconnect, and $\emptyset\neg\{i\}$:arrive(m)—synchronize with the environment (giving E ξ ξ') but appear to 'stutter' (requiring F ξ ξ') relative to the underlying process.

The lifting rule for partial networks is the most demanding. The function net-tree-ips, giving the set of addresses in a net-tree, plays a key role.

Lemma 5 (opnet lifting). *Given* $(\sigma, s \parallel t) \in$ *oreachable (opnet onp* $(p_1 \parallel p_2)$*)* S U, *where* $S =$ *otherwith* E *(net-tree-ips* $(p_1 \parallel p_2)$*) (oarrivemsg* I*),* $U =$ *other* F *(net-tree-ips* $(p_1 \parallel p_2)$*), and* E *and* F *are reflexive, for arbitrary* $p\ i$ *of the form* $\langle i : \text{onp}\ i : R \rangle_o$, $p\ i \models (\lambda\sigma\ \text{-}.\ \text{oarrivemsg}\ I\ \sigma, \text{other}\ F\ \{i\} \rightarrow) (\lambda((\sigma, \text{-}), a, (\sigma', \text{-})).\ \text{castmsg}\ (I\ \sigma)\ a)$, *and similar step invariants for* $E\ (\sigma\ i)\ (\sigma'\ i)$ *and* $F\ (\sigma\ i)\ (\sigma'\ i)$, *then it follows that both* $(\sigma, s) \in$ *oreachable (opnet onp* p_1*)* S_1 U_1 *and* $(\sigma, t) \in$ *oreachable (opnet onp* p_2*)* S_2 U_2, *where* S_1 *and* U_1 *are over* p_1, *and* S_2 *and* U_2 *are over* p_2.

The proof is by induction over oreachable. The initial and interleaved cases are trivial. For the local case, given open reachability of (σ, s) and (σ, t) for p_1 and p_2, respectively, and $((\sigma, s \parallel t), a, (\sigma', s' \parallel t')) \in$ trans (opnet onp $(p_1 \parallel p_2)$), we must show open reachability of (σ', s') and (σ', t'). The proof proceeds by cases of a. The key step is to have stated the lemma without introducing cyclic dependencies between (synchronizing) assumptions and (step invariant) guarantees. For a synchronizing action like arrive, Definition 9 requires satisfaction of S_1 to advance in p_1 and of S_2 to advance in p_2, but the assumption S only holds for addresses $j \notin$ net-tree-ips $(p_1 \parallel p_2)$. This is why the step invariants required of nodes only assume oarrivemsg I σ of the environment, rather than an S over node address {i}.

This is not unduly restrictive since the step invariants provide guarantees for individual local state elements and not between network nodes. The assumption oarrivemsg l σ is never cyclic: it is either assumed of the environment for paired arrives, or trivially satisfied for the side that *casts. The step invariants are lifted from nodes to partial networks by induction over net-trees. For non-synchronizing actions, we exploit the extra guarantees built into the open SOS rules.

The rule for closed networks is similar to the others. Its important function is to eliminate the synchronizing assumption (S in the lemmas above), since messages no longer arrive from the environment. The conclusion of this rule has the form required by the transfer lemma of the next section.

4.4 Transferring Open Invariants

The rules in the last section extend invariants over sequential processes, like that of (7), to arbitrary, open network models. All that remains is to transfer the extended invariants to the standard model. We do so using a locale [12] openproc np onp sr where np has type ip ⇒ ('s, 'm seq-action) automaton, onp has type ip ⇒ ((ip ⇒ 'g) × 'l, 'm seq-action) automaton, and sr has type 's ⇒ 'g × 'l. The automata use the actions of Section 2.1 with arbitrary messages ('m seq-action).

The openproc locale relates an automaton np to a corresponding 'open' automaton onp, where sr splits the states of the former into global and local components. Besides two technical conditions on initial states, this relation requires assuming σ i = fst (sr s), σ' i = fst (sr s') and (s, a, s') ∈ trans (np i), and then showing ((σ, snd (sr s)), a, (σ', snd (sr s'))) ∈ trans (onp i)—that is, that onp simulates np. For our running example, we show openproc ptoy optoy id, and then lift it to the composition with qmsg, using a generic relation on openproc locales.

Lemma 6 (transfer). *Given* np, onp, *and* sr *such that* openproc np onp sr, *then for any* wf-net-tree n *and* s ∈ reachable (closed (pnet np n)) (λ-. True), *it follows that* (default (someinit np sr) (netlift sr s), netliftl sr s)
∈ oreachable (oclosed (opnet onp n)) (λ- - -. True) U.

This lemma uses two openproc constants: someinit np sr i chooses an arbitrary initial state from np (SOME x. x ∈ (fst ∘ sr) ' init (np i)), and

$$\begin{aligned} \text{netliftl sr } (s^{i}_{R}) &= (\text{snd (sr s)})^{i}_{R} \\ \text{netliftl sr } (s \parallel t) &= (\text{netliftl sr s}) \parallel (\text{netliftl sr t}). \end{aligned}$$

The proof of the lemma 'discharges' the assumptions incorporated into the open SOS rules. An implication from an open invariant on an open model to an invariant on the corresponding standard model follows as a corollary.

Summary. The technicalities of the lemmas in this and the preceding section are essential for the underlying proofs to succeed. The key idea is that through an open version of AWN where automaton states are segregated into global and local components, one can reason locally about global properties, but still, using the so called transfer and lifting results, obtain a result over the original model.

5 Concluding Remarks

We present a mechanization of a modelling language for MANET and WMN protocols, including a streamlined adaptation of standard theory for showing invariants of individual reactive processes, and a novel and compositional framework for lifting such results to network models. The framework allows the statement and proof of inter-node properties. We think that many elements of our approach would apply to similarly structured models in other formalisms.

It is reasonable to ask whether the basic model presented in Section 2 could not simply be abandoned in favour of the open model of Section 4.1. But we believe that the basic model is the most natural way of describing what AWN means, proving semantic properties of the language, showing 'node-only' invariants, and, potentially, for showing refinement relations. Having such a reference model allows us to freely incorporate assumptions into the open SOS rules, knowing that their soundness must later be justified.

The Ad hoc On-demand Distance Vector (AODV) case study. The framework we present in this paper was successfully applied in the mechanization of a proof of loop freedom [6, §7] of the AODV protocol [17], a widely-used routing protocol designed for MANETs, and one of the four protocols currently standardized by the IETF MANET working group. The model has about 100 control locations across 6 different processes, and uses about 40 functions to manipulate the data state. The main property (loop freedom) roughly states that 'a data packet is never sent round in circles without being delivered'. To establish this property, we proved around 400 lemmas. Due to the complexity of the protocol logic and the length of the proof, we present the details elsewhere [4]. The case study shows that the presented framework can be applied to verification tasks of industrial relevance.

Acknowledgments. We thank G. Klein and M. Pouzet for support and complaisance, and M. Daum for participation in discussions. Isabelle/jEdit [21], Sledgehammer [2], parallel processing [22], and the TPTP project [19] were invaluable.

NICTA is funded by the Australian Government through the Department of Communications and the Australian Research Council through the ICT Centre of Excellence Program.

References

1. Bengtson, J., Parrow, J.: Psi-calculi in Isabelle. In: Berghofer, S., Nipkow, T., Urban, C., Wenzel, M. (eds.) TPHOLs 2009. LNCS, vol. 5674, pp. 99–114. Springer, Heidelberg (2009)
2. Blanchette, J.C., Böhme, S., Paulson, L.C.: Extending Sledgehammer with SMT solvers. In: Bjørner, N., Sofronie-Stokkermans, V. (eds.) CADE 2011. LNCS (LNAI), vol. 6803, pp. 116–130. Springer, Heidelberg (2011)
3. Bourke, T.: Mechanization of the Algebra for Wireless Networks (AWN). In: Archive of Formal Proofs (2014), `http://afp.sf.net/entries/AWN.shtml`

4. Bourke, T., van Glabbeek, R.J., Höfner, P.: A mechanized proof of loop freedom of the (untimed) AODV routing protocol. See authors' webpages (2014)
5. Chaudhuri, K., Doligez, D., Lamport, L., Merz, S.: Verifying safety properties with the TLA$^+$ proof system. In: Giesl, J., Hähnle, R. (eds.) IJCAR 2010. LNCS, vol. 6173, pp. 142–148. Springer, Heidelberg (2010)
6. Fehnker, A., van Glabbeek, R.J., Höfner, P., McIver, A., Portmann, M., Tan, W.L.: A process algebra for wireless mesh networks used for modelling, verifying and analysing AODV. Technical Report 5513, NICTA (2013), http://arxiv.org/abs/1312.7645
7. Feliachi, A., Gaudel, M.-C., Wolff, B.: Isabelle/Circus: A process specification and verification environment. In: Joshi, R., Müller, P., Podelski, A. (eds.) VSTTE 2012. LNCS, vol. 7152, pp. 243–260. Springer, Heidelberg (2012)
8. Fokkink, W., Groote, J.F., Reniers, M.: Process algebra needs proof methodology. EATCS Bulletin 82, 109–125 (2004)
9. Göthel, T., Glesner, S.: An approach for machine-assisted verification of Timed CSP specifications. Innovations in Systems and Software Engineering 6(3), 181–193 (2010)
10. Heyd, B., Crégut, P.: A modular coding of UNITY in COQ. In: Goos, G., Hartmanis, J., van Leeuwen, J., von Wright, J., Grundy, J., Harrison, J. (eds.) TPHOLs 1996. LNCS, vol. 1125, pp. 251–266. Springer, Heidelberg (1996)
11. Hirschkoff, D.: A full formalisation of π-calculus theory in the Calculus of Constructions. In: Gunter, E.L., Felty, A.P. (eds.) TPHOLs 1997. LNCS, vol. 1275, pp. 153–169. Springer, Heidelberg (1997)
12. Kammüller, F., Wenzel, M., Paulson, L.C.: Locales - A sectioning concept for Isabelle. In: Bertot, Y., Dowek, G., Hirschowitz, A., Paulin, C., Théry, L. (eds.) TPHOLs 1999. LNCS, vol. 1690, pp. 149–165. Springer, Heidelberg (1999)
13. Manna, Z., Pnueli, A.: Temporal Verification of Reactive Systems: Safety. Springer (1995)
14. Müller, O.: A Verification Environment for I/O Automata Based on Formalized Meta-Theory. PhD thesis, TU München (1998)
15. Nipkow, T., Paulson, L.C., Wenzel, M.: Isabelle/HOL. LNCS, vol. 2283. Springer, Heidelberg (2002)
16. Paulson, L.C.: The inductive approach to verifying cryptographic protocols. J. Computer Security 6(1-2), 85–128 (1998)
17. Perkins, C.E., Belding-Royer, E.M., Das, S.R.: Ad hoc on-demand distance vector (AODV) routing. RFC 3561 (Experimental), Network Working Group (2003)
18. de Roever, W.-P., de Boer, F., Hannemann, U., Hooman, J., Lakhnech, Y., Poel, M., Zwiers, J.: Concurrency Verification: Introduction to Compositional and Noncompositional Methods. Cambridge Tracts in Theor. Comp. Sci., vol. 54. CUP (2001)
19. Sutcliffe, G.: The TPTP problem library and associated infrastructure: The FOF and CNF parts, v3.5.0. J. Automated Reasoning 43(4), 337–362 (2009)
20. Tej, H., Wolff, B.: A corrected failure divergence model for CSP in Isabelle/HOL. In: Fitzgerald, J.S., Jones, C.B., Lucas, P. (eds.) FME 1997. LNCS, vol. 1313, pp. 318–337. Springer, Heidelberg (1997)
21. Wenzel, M.: Isabelle/jEdit – A prover IDE within the PIDE framework. In: Jeuring, J., Campbell, J.A., Carette, J., Dos Reis, G., Sojka, P., Wenzel, M., Sorge, V. (eds.) CICM 2012. LNCS, vol. 7362, pp. 468–471. Springer, Heidelberg (2012)
22. Wenzel, M.: Shared-memory multiprocessing for interactive theorem proving. In: Blazy, S., Paulin-Mohring, C., Pichardie, D. (eds.) ITP 2013. LNCS, vol. 7998, pp. 418–434. Springer, Heidelberg (2013)